STUDY SESSION

# 楕円曲線暗号

## 実装を通して学ぼう

wuhu1sland (49th)

CRYPTOGRAPHY OVERVIEW

# 暗号技術の全体像

## 共通鍵暗号

**AES** — Advanced Encryption Standard

Wi-Fi, SSH (暗号化) などで使用

**ChaCha20** — Stream Cipher

Wireguard (暗号化) などで使用

**主な用途:** 暗号化

## 公開鍵暗号

**RSA** — Rivest-Shamir-Adleman

HTTPS, SSH, VPN (署名、鍵交換)

**ECC** — Elliptic Curve Cryptography

HTTPS, SSH, VPN (署名、鍵交換)

**主な用途:** 署名、鍵交換、認証

### 署名とは

メッセージの送信者を証明し、改ざんを検出する技術

### 鍵交換とは

事前の秘密共有なしに、安全に暗号鍵を共有する技術

SECURITY FOUNDATION

# RSAとECCの違い

安全性の根拠とアルゴリズムの比較

## RSA 素因数分解の困難性

大きな合成数を素因数分解することの計算困難性に基づく

$n = p \times q$ (p, q: 大きな素数)

RSAのアルゴリズム詳細は各自調査を

## ECC 離散対数問題の困難性

素数 p と p 未満の自然数 g, x を用いて

$g^x \equiv m \pmod{p}$

g, p, m が分かっても x を求めることが困難

楕円曲線上の離散対数問題を利用

## ECCの代表的なアルゴリズム

| ECDH | ECDSA | Schnorr | EdDSA |
|---|---|---|---|
| 鍵交換 | 署名 | 署名 | 署名 |

MATHEMATICAL FOUNDATION

# 楕円曲線の数学的基礎

## 楕円曲線の定義

Weierstrassの標準形

$$y^2 = x^3 + ax + b$$

Edwards曲線

$$ax^2 + y^2 = 1 + dx^2y^2$$

パラメータの値はアルゴリズムごとに異なります

## 代表的な曲線例

**secp256k1** Bitcoinで使用

$$y^2 = x^3 + 7 \ (a=0, b=7)$$

**Ed25519** EdDSAで使用

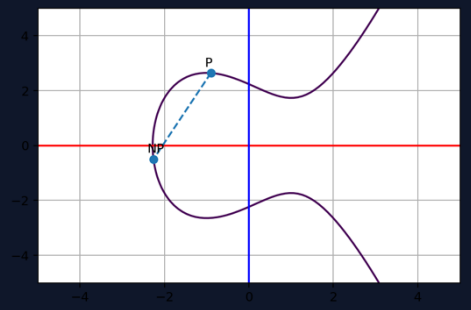
$$-x^2 + y^2 = 1 - (121665/121666)x^2y^2$$

パラメータ: 法 p、ベースポイント G、Gの位数 n

## 有限体上の楕円曲線

素数 p を法とする整数の剰余集合 {0, 1, ..., p-1} に対して、四則演算が閉じている（0以外の任意の元に対して乗法の逆元が存在）ため、これを**有限体 Fp**と呼びます。

有限体上では曲線は**ランダムに見える点の集合**となります



ELLIPTIC CURVE OPERATIONS

# 楕円曲線上の演算

点の加算と群の構造

## 点の加算（幾何的イメージ）

1. 2点 P と Q を通る直線を引く
2. その直線と曲線の3つ目の交点を求める
3. その点をx軸対称に反転させた点が P + Q

参考: YouTube - Elliptic Curve Animation

## 群の構造

無限遠点を単位元として曲線に含めることで、楕円曲線は群になります。

**単位元**
無限遠点

**逆元**
x軸対称の点

**結合法則**
(P + Q) + R = P + (Q + R)

## 離散対数問題

順に計算を繰り返せば $g^x \equiv m$ は計算できるが、m と g が分かっても x が復元できないという性質が、楕円曲線暗号の安全性の根拠となります。

KEY EXCHANGE

# ECDH: 楕円曲線Diffie-Hellman鍵交換

安全な共有秘密の作成

## 0 準備

AliceとBobで事前に共有

- 楕円曲線のパラメータ
- ベースポイント G
- Gの位数 n

## 1 鍵生成

**Alice**

秘密鍵: $d_A \in [1, n-1]$

公開鍵: $Q_A = d_A G$

**Bob**

秘密鍵: $d_B \in [1, n-1]$

公開鍵: $Q_B = d_B G$

⇄ 公開鍵を交換

## 2 共有秘密

**Aliceの計算**

$K = d_A Q_B$

**Bobの計算**

$K = d_B Q_A$

$K = d_A d_B G$

同じ値に到達!

### ポイント

自分の**秘密鍵を明かすことなく**、同じ値 K を共有できた

### セキュリティ

通信が傍受されても、$d_A$ と $d_B$ のどちらも知らない第三者は K を作れない

K（楕円曲線上の点）のx座標を共通鍵の素とすれば、信頼できない通信路上でも安全に二者間の共通鍵を作成可能

DIGITAL SIGNATURE

# ECDSA: 楕円曲線デジタル署名アルゴリズム

SSL証明書やBitcoinで広く使用される署名のスタンダード

## 署名生成 (Alice)

1. 秘密鍵 $d_A$ と公開鍵 $Q_A = d_A G$ を用意
2. メッセージ m のハッシュ $e = H(m)$ を計算
3. e の上位をマスクして z を作成
4. 乱数 $k \in [1, n-1]$ を生成（nonce）
   $(x, y) = kG$ を計算
5. $r = x \bmod n$
   $s = k^{-1}(z + rd_A) \bmod n$
6. 署名 (r, s) を送信

## 署名検証 (Bob)

7. 署名 (r, s) を受け取る
8. 同様にハッシュ値 e とマスク z を計算
9. $u_1 = zs^{-1} \bmod n$
   $u_2 = rs^{-1} \bmod n$
10. $(x', y') = u_1 G + u_2 Q_A$
11. $x' \equiv r \pmod n$
    → 正当な署名！

⚠ nonce kは内部的な秘密鍵。同じkを使い回すと危険！

ℹ 1999年にNISTにより標準化された署名アルゴリズム

CASE STUDY

# 教訓: PS3事件とnonceの重要性

2010年に発生したECDSAの実装ミスによる大事件

## PS3の脆弱性

2010年

SONYはプレステ3のソフトウェア署名にECDSAを採用していたが、**全ての署名で同じnonce kを使い回して**しまった。

**結果**

George Hotzを始めとするハッカーコミュニティが、脆弱性から**署名用秘密鍵dAを逆算**することに成功

## なぜ秘密鍵が漏れるのか

ECDSAでは署名(r, s)の$r = x \bmod n$（ここで(x,y) = kG）は**kだけに依存**します。

同じkを2回使うと、rは同じ値になります。

## 数学的解析

2つの異なるメッセージ$m_1$, $m_2$に対して、同じkで署名を作ると:

$$s_1 = k^{-1}(z_1 + rd_A) \bmod n$$

$$s_2 = k^{-1}(z_2 + rd_A) \bmod n$$

未知の変数はkと$d_A$の2つだが、方程式が2つあるため解けます:

$$k \equiv (z_1 - z_2) / (s_1 - s_2) \bmod n$$

## 鍵を割った人物

**fail0verflowチーム**
marcan, bushing, sven
固定nonceの特定に成功

**George Hotz**
ソフトウェア署名用の鍵を割った
17歳でiPhone jailbreakに成功したハッカー

ADVANCED SIGNATURES

# Schnorr署名とEdDSA

より効率的で安全な署名アルゴリズム

## S Schnorr署名

1989年発明

### 署名生成

1. 乱数 r を用いて R = rG を計算
2. メッセージ m と R を結合 (R ‖ m)
3. $s = r + H(R \| m)d_A$
4. 署名 (R, s) を送信

### 署名検証

$sG = R + H(R \| m)Q_A$

成立すれば正当な署名

★ めちゃくちゃシンプル！ 証明も簡単

## Ed EdDSA

Edwards-curve DSA

### 特徴1: 決定論的nonce

秘密鍵のハッシュから決定論的に計算

→ 乱数生成器の安全性に依存しない

### 特徴2: cofactor 8

検証時に$8(SG) = 8(R + kQ_A)$を確認

twisted Edwards曲線の位数 n = 8L のため

### 完全加法公式

点の加算に場合分けが不要→ サイドチャネル攻撃に強い

### Schnorr vs ECDSA

Schnorr署名は検証効率が良く、集約署名が可能。特許が切れてからECDSAに代わる方式として注目され、BitcoinのTaprootで採用された。

REAL-WORLD APPLICATIONS

# 応用例: SSHとBitcoin

楕円曲線暗号の実際の使用例

## SSH

1. **鍵交換**
   共有秘密を作成し、共通鍵を生成
2. **サーバ認証**
   ホスト鍵で自分を証明、known_hostsと照合
3. **クライアント認証**
   チャレンジに秘密鍵で署名して返す
4. **検証**
   公開鍵で検証、対応する公開鍵があれば認証

## Bitcoin

### 支払いの流れ

1. UTXOを秘密鍵で署名してアンロック
2. 署名と公開鍵つきトランザクションを作成
3. マイナーが署名を検証
4. PoWでブロックを作成し追加

### 使用アルゴリズム

secp256k1 ECDSA

### Taprootアップグレード (2021)

Schnorr署名を導入。線形性を活用したマルチシグやバッチ検証が可能に。

TRUST & TRANSPARENCY

# Snowden事件と暗号の信頼性

暗号標準の透明性の重要性

## Snowden事件

2013年

映画にもなった有名な事件。NSAによる大規模な監視活動の告発が発端となった。

**核心となる疑惑**

NIST-p256などの楕円曲線の乱数生成器としてNSAが推奨していたDual_EC_DRBGというアルゴリズムに、**バックドアが仕込まれていたのではないか**という疑惑

## 技術的な問題点

Dual_EC_DRBGでは、二つのベースポイントの離散対数問題が予測不可能性の根拠となっている。

ベースポイントの決定根拠が**不明瞭**で、NSAが事前に解となるスカラー値を把握しているポイントをあえて設定したのではないか、と疑われた。

## 脆弱性の影響

**乱数生成の問題**

生成パターンが予想可能になると、nonceなどの値が割れてしまう

**秘密鍵の危険性**

結果的に**秘密鍵の復元**などに使われかねない

## 教訓

- ✓ 暗号標準の**透明性**が重要
- ✓ パラメータの決定根拠を**公開**すべき
- ✓ オープンな**レビューと検証**が必要

参考: Dual_EC_DRBGの脆弱性詳細

EXERCISE & SUMMARY

# 演習とまとめ

## 演習

EdDSA（またはSchnorr署名）を実装してみよう

- 鍵ペアの生成
- 署名の作成
- 署名の検証

## 学習ポイント

- 楕円曲線の数学的基礎
- ECDH鍵交換の仕組み
- ECDSA署名の生成と検証
- nonceの重要性と危険性
- Schnorr/EdDSAの利点

### 楕円曲線暗号の本質

離散対数問題の困難性を利用し、**短い鍵長で高い安全性**を実現。現代のインターネット通信や暗号通貨の基盤となっている重要技術。